PaintBot

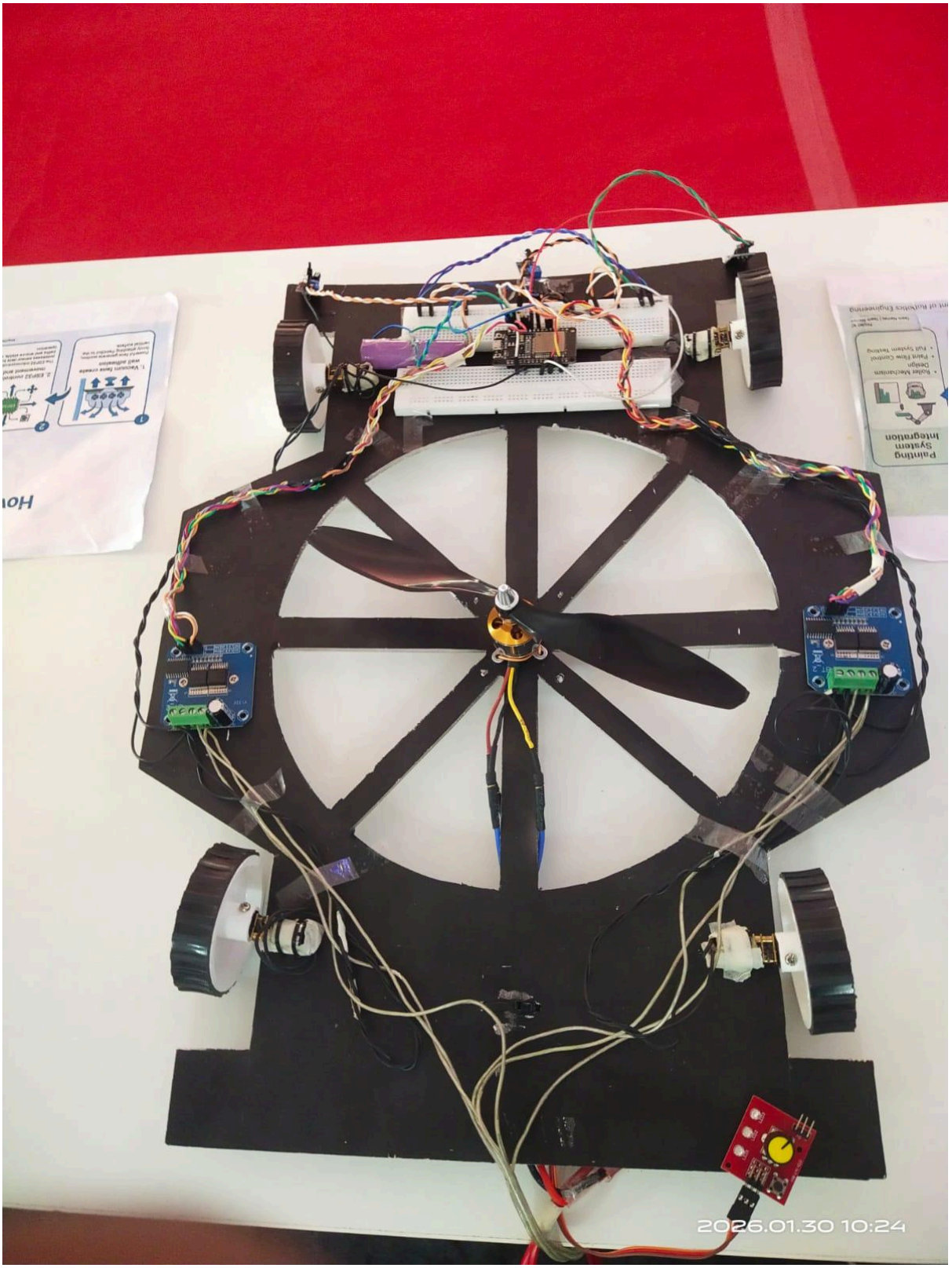
Components Used

1. 4 N20 meter
2. 2 Motor driver
3. 4 Wheels
4. 1 Esp 32
5. 3 IR sensor
6. 1 Bldc Motor
7. 1 10inch Propeller
8. 1 breadboard
9. 1 Air compressor
10. 1 Dc submersible pump
11. Air compressor pipe
12. Esc (Bldc Motor controller)
13. 2 Power supply
14. Servo tester
15. Material (for body)- sunboard
16. Jumper wire (20cm)

Abstract:

PaintBot is an innovative robotic system designed to automate interior wall painting to improve efficiency, safety, and consistency in construction projects. The system uses intelligent navigation, precision spray technology, and sensor-based obstacle detection to deliver uniform paint coverage while avoiding windows, switchboards, and other fixtures. The interior robot uses a suction-based system and a wired power connection to reduce battery dependency and prevent falls. By minimizing manual high-risk labor and enabling faster, continuous operation, PaintBot aims to reduce project time, improve painting quality, and lower overall construction costs for builders.

Model Photo :


2026.01.30 10:24

Photos of sensors Used:

Ir sensor

[redacted]

[redacted]

Program Code:

```
#include <Arduino.h>

// ———————— IR SENSORS ————————
#define IR_1 34   // Front obstacle (LOW = detected)
#define IR_L 27   // Left IR (LOW = detected)
#define IR_R 4    // Right IR (LOW = detected)
#define A 18
#define B 19
#define C 23

// ———————— BTS7960 PINS ————————
// Right motor driver
#define R_RPWM 21
#define R_LPWM 22
#define R_EN_R 25
#define R_EN_L 32

// Left motor driver
#define L_RPWM 13
#define L_LPWM 15
#define L_EN_R 14
#define L_EN_L 12

const int SPEED_PWM = 180;
bool turning = false;        // LEFT latch (your existing one)
bool turningRight = false;   // NEW: RIGHT latch



// ———————— PWM HELPERS ————————
static inline void pwmWritePin(uint8_t pin, uint8_t duty) {
  ledcWrite(pin, duty);
}

static inline void pwmAttachPin(uint8_t pin, uint32_t freq = 20000, uint8_t res = 8) {
  ledcAttach(pin, freq, res);
}
```

```
// ———————— MOTOR CONTROL ————————
void enableDrivers() {
  pinMode(R_EN_R, OUTPUT); pinMode(R_EN_L, OUTPUT);
  pinMode(L_EN_R, OUTPUT); pinMode(L_EN_L, OUTPUT);

  digitalWrite(R_EN_R, HIGH);
  digitalWrite(R_EN_L, HIGH);
  digitalWrite(L_EN_R, HIGH);
  digitalWrite(L_EN_L, HIGH);
}

void stopMotors() {
  pwmWritePin(R_RPWM, 0);
  pwmWritePin(R_LPWM, 0);
  pwmWritePin(L_RPWM, 0);
  pwmWritePin(L_LPWM, 0);
}

// Rotate left
void Rl() {
  pwmWritePin(R_RPWM, SPEED_PWM);
  pwmWritePin(R_LPWM, 0);
  pwmWritePin(L_RPWM, 0);
  pwmWritePin(L_LPWM, SPEED_PWM);
}

// Rotate right
void Rr() {
  pwmWritePin(R_RPWM, 0);
  pwmWritePin(R_LPWM, SPEED_PWM);
  pwmWritePin(L_RPWM, SPEED_PWM);
  pwmWritePin(L_LPWM, 0);
}


// Forward motion
void forwardMotors() {
  // Left forward
  pwmWritePin(L_RPWM, SPEED_PWM);
  pwmWritePin(L_LPWM, 0);

  // Right forward
  pwmWritePin(R_RPWM, SPEED_PWM);
  pwmWritePin(R_LPWM, 0);
}

void setup() {
```

```
  Serial.begin(115200);

  pinMode(IR_1, INPUT);
  pinMode(IR_L, INPUT);
  pinMode(IR_R, INPUT);

  // Switches WITHOUT pull-up or pull-down
  pinMode(A, INPUT);
  pinMode(B, INPUT);
  pinMode(C, INPUT);

  pwmAttachPin(R_RPWM);
  pwmAttachPin(R_LPWM);
  pwmAttachPin(L_RPWM);
  pwmAttachPin(L_LPWM);

  enableDrivers();
  stopMotors();
}

void loop() {
  // Read IR (LOW = detected)
  bool obsDetected = (digitalRead(IR_1) == LOW);

  // Read switches
  bool switch_A = (digitalRead(A) == HIGH);
  bool switch_B = (digitalRead(B) == HIGH);
  bool switch_C = (digitalRead(C) == HIGH);

  // ----------------------------
  // HIGHEST PRIORITY STOP
  // If LEFT or RIGHT IR is HIGH -> STOP EVERYTHING
  // ----------------------------
  if (digitalRead(IR_L) == HIGH || digitalRead(IR_R) == HIGH) {
    turning = false;
    turningRight = false;
    stopMotors();
    delay(10);
    return;
  }



  // ----------------------------
  // 1) RIGHT-TURN TRIGGER (A mode)
  // A=1, B=0, C=0 AND obstacle detected -> start right latch
  // ----------------------------
  if (!turningRight && !turning && switch_A && !switch_B && !switch_C && obsDetected) {
```

```
  turningRight = true;
}

// ----------------------------
// 2) RIGHT-TURN LATCH
// keep rotating right until: B=1, A=0, C=0
// ----------------------------
if (turningRight) {
  if (!switch_A && switch_B && !switch_C) {
    turningRight = false;
    stopMotors();
  } else {
    Rr(); // keep rotating regardless of IR
  }
  delay(10);
  return;
}

// ----------------------------
// 3) LEFT-TURN TRIGGER (B mode)
// B=1, A=0, C=0 AND obstacle detected -> start left latch
// ----------------------------
if (!turning && !turningRight && !switch_A && switch_B && !switch_C && obsDetected) {
  turning = true;
}

// ----------------------------
// 4) LEFT-TURN LATCH
// keep rotating left until: A=1, B=0, C=0
// ----------------------------
if (turning) {
  if (switch_A && !switch_B && !switch_C) {
    turning = false;
    stopMotors();
  } else {
    Rl(); // keep rotating regardless of IR
  }
  delay(10);
  return;
}

// ----------------------------
// 5) FORWARD RULES (only when not turning)
// A=1,B=0,C=0 and NO obstacle -> forward
// B=1,A=0,C=0 and NO obstacle -> forward
// else -> stop
// ----------------------------
if (!obsDetected) {
```

```
    if ((switch_A && !switch_B && !switch_C) || (!switch_A && switch_B && !switch_C)) {
      forwardMotors();
    } else {
      stopMotors();
    }
  } else {
    // obstacle detected but not matching any start condition (or invalid switch combo)
    stopMotors();
  }

  delay(10);
}
```